Tras ver cómo trata Python el paso por valor y por referencia de parámetros y su relación con objetos mutables e inmutables ejecuta los fragmentos de código dados (ver aula virtual):

1. Depúralos para ver qué resultado se obtiene en cada uno de los casos.
2. Argumenta el porqué de cada resultado.

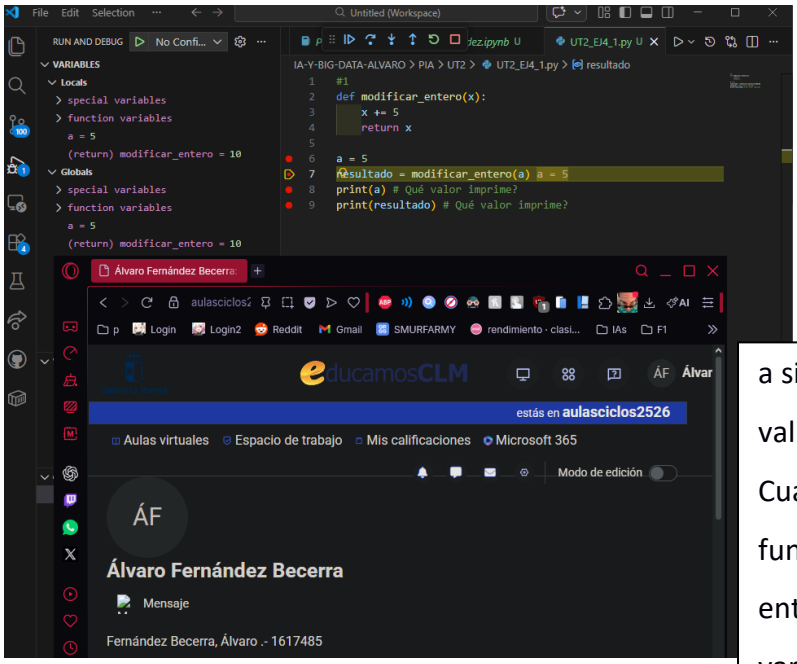
Realiza la tarea indicada para los ejercicios del 1 al 7 (Los de clases los comentaremos entre todos)

Ejercicios:

1.

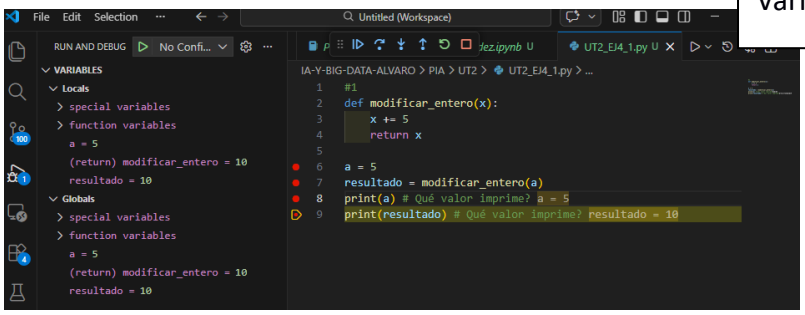
```python
#1
def modificar_entero(x):
    x += 5
    return x

a = 5
resultado = modificar_entero(a)
print(a)  # ¿Qué valor se imprime?
print(resultado)  # ¿Qué valor se imprime?
```



a sigue

valiendo 5 porque **int** es **inmutable**. Cuando se hace x += 5 dentro de la función, Python crea un nuevo objeto entero con valor 10 y lo asigna a la variable local x

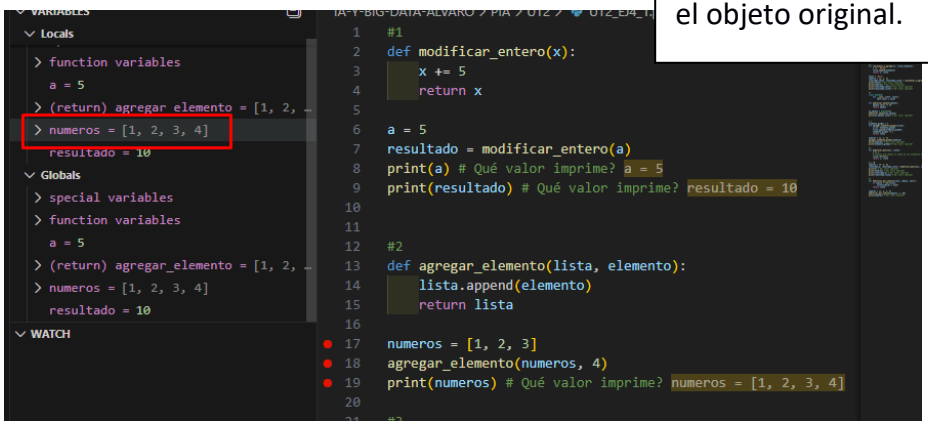


2.

```python
#2
def agregar_elemento(lista, elemento):
    lista.append(elemento)
    return lista

numeros = [1, 2, 3]
agregar_elemento(numeros, 4)
print(numeros)  # ¿Qué valor se imprime?
```

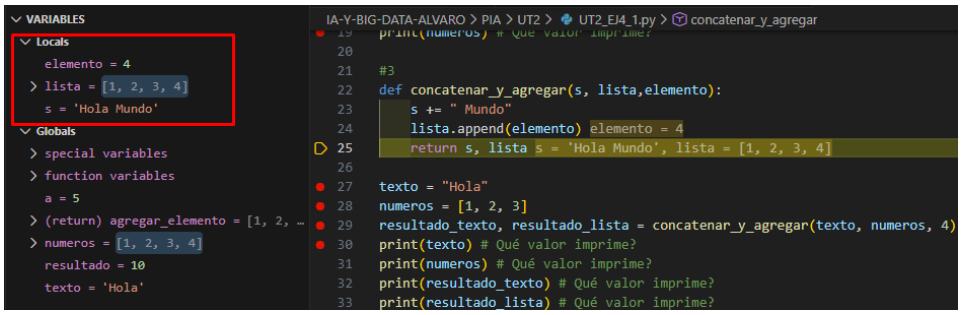
Las listas **son** mutables. La función recibe una referencia al mismo objeto lista, por lo que se modifica directamente el objeto original.

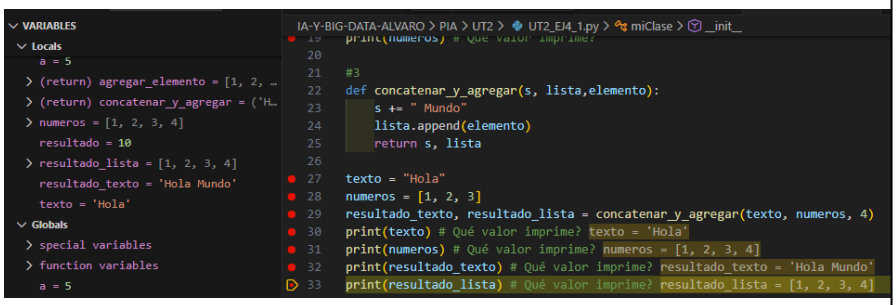


3.

```python
#3
def concatenar_y_agregar(s, lista, elemento):
    s += " mundo"
    lista.append(elemento)
    return s, lista

texto = "Hola"
numeros = [1, 2, 3]
resultado_texto, resultado_lista = concatenar_y_agregar(texto, numeros, 4)
print(texto)  # ¿Qué valor se imprime?
print(numeros)  # ¿Qué valor se imprime?
print(resultado_texto) # ¿Qué valor se imprime?
print(resultado_lista) # ¿Qué valor se imprime?
```





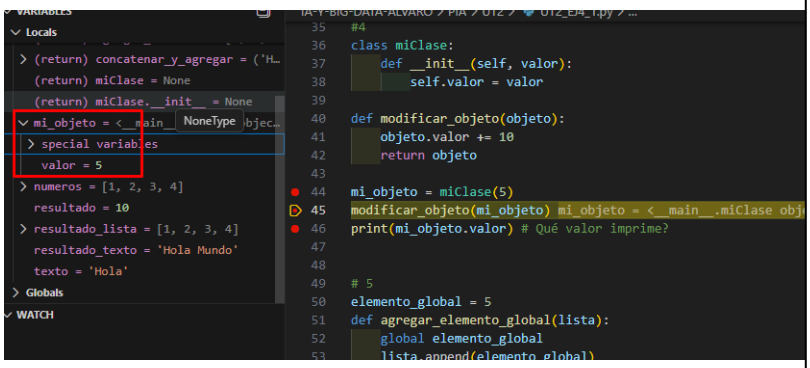
texto no cambia porque **str** es **inmutable**. s += " Mundo" crea un nuevo string local. En cambio, numeros sí cambia porque la lista es **mutable** y append actúa sobre el objeto original.

4.

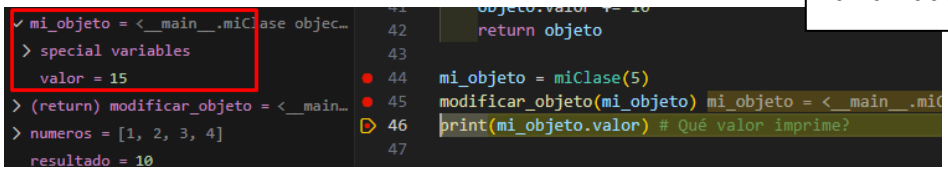
```python
#4
class MiClase:
    def __init__(self, valor):
        self.valor = valor

def modificar_objeto(objeto):
    objeto.valor += 10
    return objeto

mi_objeto = MiClase(5)
modificar_objeto(mi_objeto)
print(mi_objeto.valor)  # ¿Qué valor se imprime?
```



Las instancias de clases son **mutables**. La función recibe la referencia al mismo objeto, y al modificar su atributo el cambio persiste fuera de la función. El objeto en memoria es el mismo antes y después de la llamada.

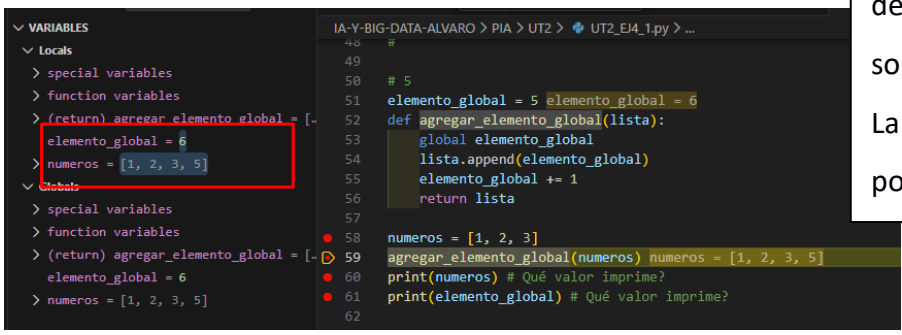


5.

```python
#5
elemento_global = 5

def agregar_elemento_global(lista):
    global elemento_global
    lista.append(elemento_global)
    elemento_global += 1
    return lista

numeros = [1, 2, 3]
agregar_elemento_global(numeros)
print(numeros)  # ¿Qué valor se imprime?
print(elemento_global)  # ¿Qué valor se imprime?
```

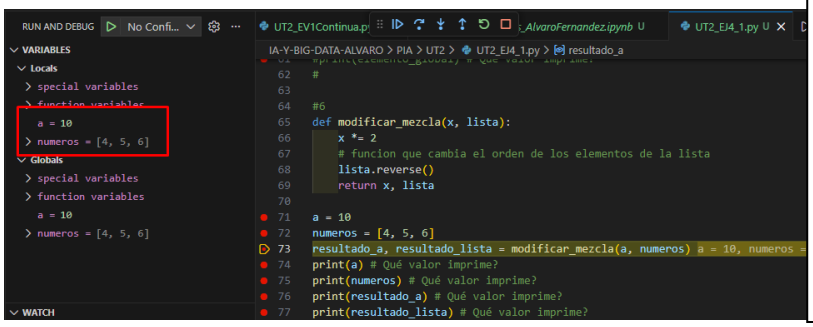


La palabra clave global permite a la función leer y modificar una variable del ámbito global. elemento_global pasa de 5 a 6 porque opera sobre la variable global real. La lista también se modifica por ser mutable

6.

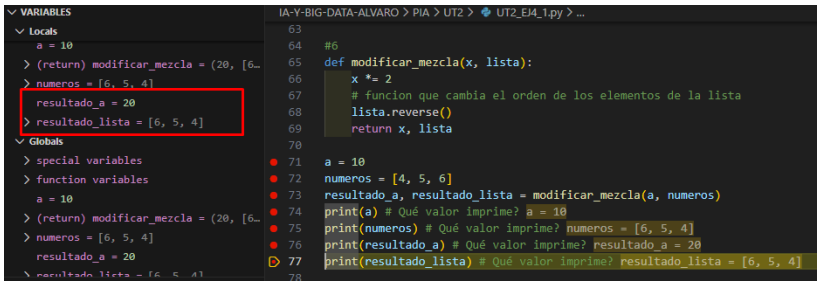
```python
#6
def modificar_mezcla(x, lista):
    x *= 2
    #funcion que cambia el orden de los elementos de la lista
    lista.reverse()
    return x, lista

a = 10
numeros = [4, 5, 6]
resultado_a, resultado_lista = modificar_mezcla(a, numeros)
print(a)  # ¿Qué valor se imprime?
print(numeros)  # ¿Qué valor se imprime?
print(resultado_a)  # ¿Qué valor se imprime?
print(resultado_lista)  # ¿Qué valor se imprime?
```



a no cambia porque x *= 2 sobre un entero crea un nuevo objeto local. En cambio, lista.reverse() invierte la lista original, por lo que numeros queda [6, 5, 4].

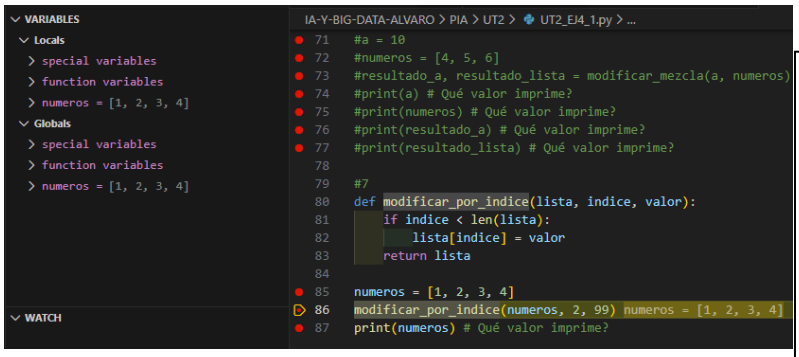
Además, resultado_lista y numeros apuntan al mismo objeto en memoria

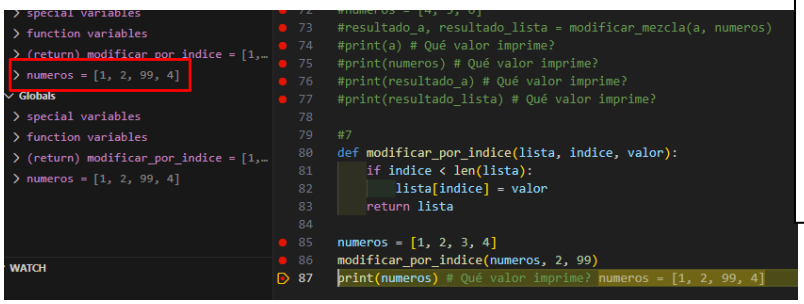


7.

```python
#7
def modificar_por_indice(lista, indice, valor):
    if indice < len(lista):
        lista[indice] = valor
    return lista

numeros = [1, 2, 3, 4]
modificar_por_indice(numeros, 2, 99)
print(numeros)  # ¿Qué valor se imprime?
```





Acceder a un elemento de lista por índice y reasignarlo modifica el objeto original directamente, exactamente igual que append o reverse. No se crea un nuevo objeto lista, sino que se actualiza una posición del objeto existente.